# Introdução

Neste guia é detalhada como configurar ambientes conda para rodar os frameworks TensorFlow e PyTorch com compatibilidade a recursos de GPU CUDA e CuDNN. É importante destacar que o tutorial foi feito pensando em sistemas operacionais Windows com placas gráficas da NVIDIA. As configurações apresentadas neste guia são apenas recomendações, sinta-se à vontade para executar outras configurações que melhor lhe atenda para desenvolver os laboratórios da capacitação.

# Guia de Configuração: TensorFlow com GPU (CUDA + CuDNN)

Este guia detalha o processo de configuração de um ambiente de desenvolvimento isolado usando a ferramenta conda para executar o TensorFlow com suporte a GPU (CUDA e CuDNN). O método a seguir é ideal para fins didáticos e de aprendizado, pois o conda gerencia automaticamente as versões do CUDA e do CuDNN, instalando-as dentro do ambiente virtual, sem conflitar com as instalações do sistema.

## Pré-requisitos

Antes de criar o ambiente, se atente se esses itens estão configurados em sua máquina:

| |
|---|
| **Miniconda**: você deve ter uma distribuição conda instalada (recomenda-se o Miniconda por ser mais leve). [Instalação do Miniconda ] |
| **Driver da Placa de Vídeo (NVIDIA)**: O conda não instala o driver da NVIDIA no seu sistema. Você deve ter esse recurso instalado em sua máquina Windows (verifique de forma simples se ele é reconhecido no seu gerenciador de tarefas). Visite o site oficial de drivers da NVIDIA para baixar e instalar o driver correto para sua GPU. |

**NOTA 1**: Provavelmente as máquinas do laboratório possuem os drivers das placas de vídeo, na maioria dos casos o sistema operacional do Windows já gerencia isso de forma automática. Instale somente se ele não estiver sendo reconhecido.

**NOTA 2**: Você não precisa instalar o “CUDA Toolkit” ou o “CuDNN” manualmente no seu sistema. O conda fará isso dentro do ambiente virtual de desenvolvimento criado. Você só precisa ter o se atentar as demandas da **NOTA 1**.

## Verificando os Drivers da Placa de Vídeo

No seu Windows, aperte as teclas “WIN+R”, digite “dxdiag” e clique em “OK”. A Figura 1 ilustra esse procedimento.

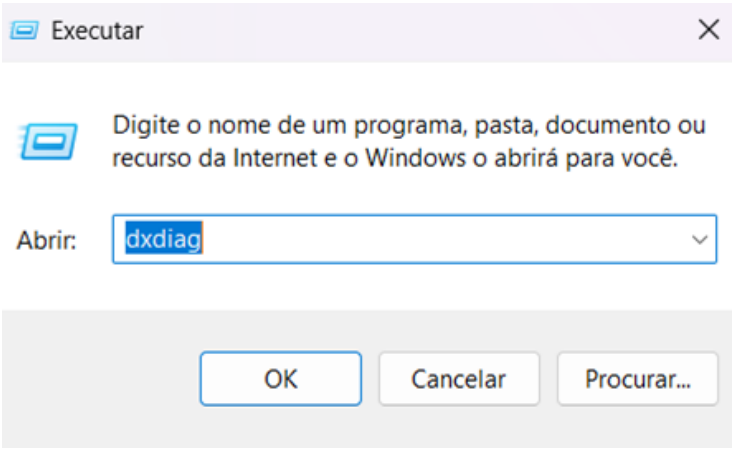



*Figura 1 – Abrindo a ferramenta de diagnóstico do DirectX para analisar a existência dos drivers da placa de vídeo.*

Em seguida, uma nova janela vai ser aberta, como ilustrado na Figura 2. Clique em “Exibição 1” ou “Exibição 2” e em algum deles você deverá ver o nome da sua placa de vídeo. No caso do exemplo, apareceu em “Exibição 2”, como ilustrado na Figura 3.

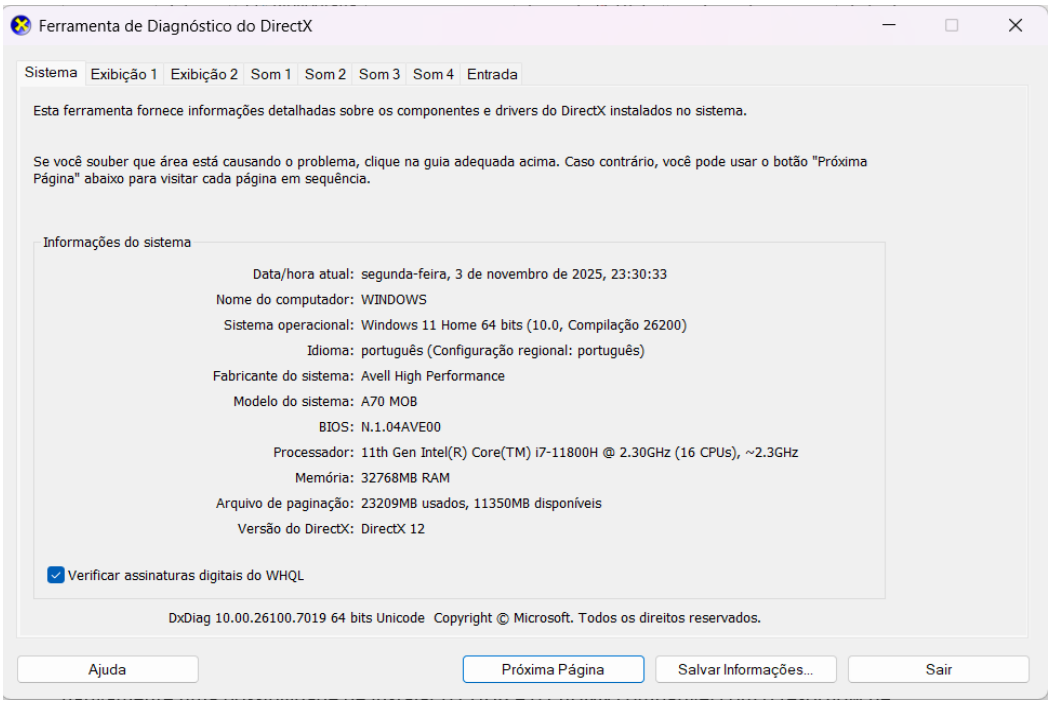



*Figura 2 – Ferramenta de diagnóstico do DirectX.*

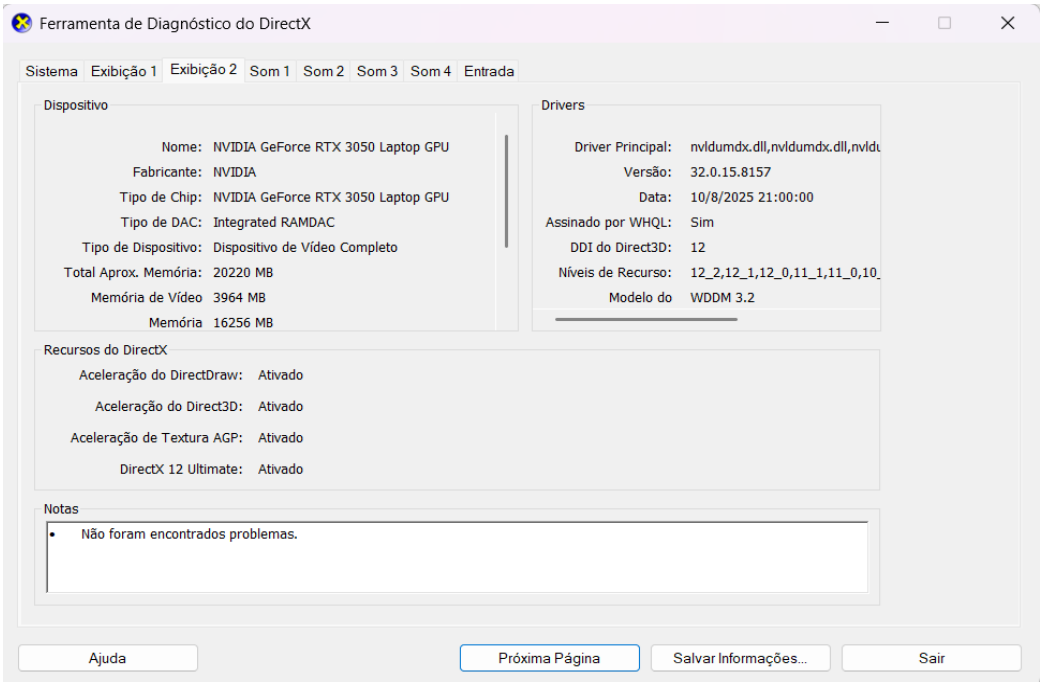



*Figura 3 – Verificando se os drivers da placa de vídeo estão instalados na máquina Windows.*

Caso não aparece o nome da sua placa de vídeo, consulte o site oficial de drivers da NVIDIA e faça as devidas instalações. Se apareceu as informações, avance para as próximas etapas.

## Criação e Configuração do Ambiente

Siga os passos a seguir no seu terminal de comandos (cmd). Primeiro, vamos criar um novo ambiente. As etapas abaixo permitem que o próprio ecossistema do conda instale as dependências no ambiente virtual do CUDA e CuDNN.

```
conda create -n tensorflow-env tensorflow-gpu

conda activate tensorflow-env
```

Assim que o processo acima for concluído, o seu prompt do terminal deve mudar, mostrando "(tensorflow-env)" no início da linha, indicando que você está dentro do ambiente Python criado. Sempre use o segundo comando para abrir esse ambiente e rodar os próximos laboratórios.

Tente rodar os comandos, LINHA A LINHA, a seguir:

```
python

import tensorflow as tf

exit()
```

Caso aconteça um erro relacionado com a versão do TensorFlow, por favor rode o seguinte comando para corrigir o conflito de versões:

```
pip install numpy==1.23.4
```

Tente rodar os comandos da etapa anterior para importar o TensorFlow novamente, o problema deve ser solucionado. Verifique também no seu terminal se o TensorFlow reconhece a GPU:

```
python

import tensorflow as tf

print(tf.config.list_physical_devices('GPU'))

exit()
```

Se for visualizado no terminal:

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Significa que o TensorFlow consegue usar a placa gráfica, todo o procedimento foi bem sucedido. Por fim, instale o ambiente do Jupyter, ferramenta comumente usados para rodar os códigos do laboratório:

```
pip install notebook
```

**NOTA 3**: Os demais pacotes auxiliares serão instalados por demanda a cada laboratório feito.

**NOTA 4**: Por favor, não tente modificar a versão do TensorFlow, Keras ou Numpy dentro desse ambiente após ter feito todas as etapas de instalação, isso irá gerar erros que a solução está por sua conta e risco.

## Guia de Configuração: PyTorch com GPU (CUDA)

Continuando o processo de instalação, o PyTorch será instalado com suporte ao CUDA em um ambiente separado do tensorflow-env. Nos laboratórios da capacitação a seguir, algumas soluções vão depender do TensorFlow e outras do PyTorch. Gerencie o uso de ambientes conforme a necessidade de uso dos *frameworks*.

Agora, vamos utilizar as facilidades do conda para instalar os pacotes do PyTorch. Semelhante ao que foi feito com a instalação do ambiente TensorFlow, o conda deve saber lidar com a versão do cudatoolkit instalada. É importante garantir que antes você não esteja dentro do ambiente de desenvolvimento Python “tensorflow-env”. Para tal, use o comando “conda deactivate” e siga com o processo de criação do ambiente “pytorch-env”:

```
conda deactivate

conda create -n pytorch-env python=3.10

conda activate pytorch-env

conda install pytorch torchvision torchaudio -c pytorch -c nvidia
```

Agora, verifique se o Pytorch tem acesso a GPU. Com o seu ambiente aberto, rode os comandos abaixo:

```
python

import torch

print(f"PyTorch is cuda available: {torch.cuda.is_available()}")

exit()
```

Para a verificação do PyTorch em específico, a variável deve retornar True caso a conexão com a placa gráfica esteja configurada.

Por fim, instale o ambiente do Jupyter neste ambiente também:

```
pip install notebook
```